МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Государственный Университет Управления

Кафедра: Институт Информационных Систем

Лабораторная работа № 8

Жадные алгоритмы и экспериментальная оценка качества приближённых решений

Дисциплина: Теория алгоритмов

Направление подготовки: 38.03.05 «Бизнес-информатика»

Язык реализации: Python 3.10+

Вариант: 14

Выполнил: студент группы БИ 2-1 Лапшенков Григорий Витальевич

Проверил: [должность, ФИО преподавателя]

[Город], 2026

1. Цель работы и задание варианта

1.1. Цель работы

Изучить жадную стратегию построения алгоритмов, установить условия, при которых она приводит к оптимальному решению, и экспериментально оценить отклонение жадного решения от оптимального в задачах, где эти условия не выполняются.

1.2. Задание варианта 14

Таблица 1. Исходные данные варианта

Параметр	Значение
Задача	выбор заявок на единственный ресурс (непересекающиеся интервалы)
Жадный критерий 1	раннее окончание
Жадный критерий 2	максимальная ценность заявки
Размерность	35 заявок
Дополнительное задание Ж7	прикладная задача: распределение ресурса между инициативами; сопоставление рекомендаций жадного и точного алгоритмов; цена ошибки в деньгах

Постановка. Заявка i задаётся тройкой (s_i, e_i, v_i): начало, окончание и ценность. Две заявки совместимы, если не пересекаются (начало следующей не раньше окончания предыдущей). Требуется выбрать набор попарно совместимых заявок с **максимальной суммарной ценностью**. Так как у критерия 2 в варианте есть ценность заявки, целевая функция — суммарная ценность. Дополнительно (для проверки теории) рассматривается классическая постановка «максимальное число заявок» — все ценности равны 1.

Данные эксперимента. 500 случайных наборов по 35 заявок: начало — случайный час на горизонте 100 ч, длительность 3...20 ч, ценность 10...100 (целое); генератор с фиксированным зерном.

2. Описание задачи, жадных критериев и точного алгоритма

2.1. Жадный алгоритм

Заявки упорядочиваются по критерию; затем они просматриваются по очереди, и заявка принимается, если не пересекается с уже принятыми. Решения не пересматриваются. В программе принятые заявки хранятся упорядоченно по началу, и проверка на пересечение сводится к двум соседям, поэтому трудоёмкость $\Theta(n \log n)$.

- **Критерий 1 — раннее окончание.** Порядок по возрастанию e_i (при равенстве — по возрастанию s_i). Ценность заявок критерием не учитывается.
- **Критерий 2 — максимальная ценность.** Порядок по убыванию v_i (при равенстве — по возрастанию e_i). Длительность и расположение заявок не учитываются.

2.2. Когда критерий «раннее окончание» оптимален

Для задачи о **максимальном числе** заявок критерий 1 оптимален (свойство жадного выбора): заявка с наименьшим окончанием входит в некоторое оптимальное решение, ведь если в оптимальном решении первой стоит другая заявка, её можно заменить заявкой с более ранним окончанием, не создав пересечений и не уменьшив числа заявок. После выбора остаётся подзадача того же вида на заявках, начинающихся не раньше e_1 (оптимальная подструктура). Если же заявки имеют разные ценности, замена может уменьшить суммарную ценность, и свойство жадного выбора нарушается — это показано в разделе 6 и подтверждено экспериментом (раздел 5).

2.3. Точный алгоритм (эталон)

Заявки упорядочены по окончанию; $p(j)$ — число заявок, закончившихся не позднее начала j-й (находится двоичным поиском). Пусть $D[j]$ — максимальная ценность, достижимая на первых j заявках:

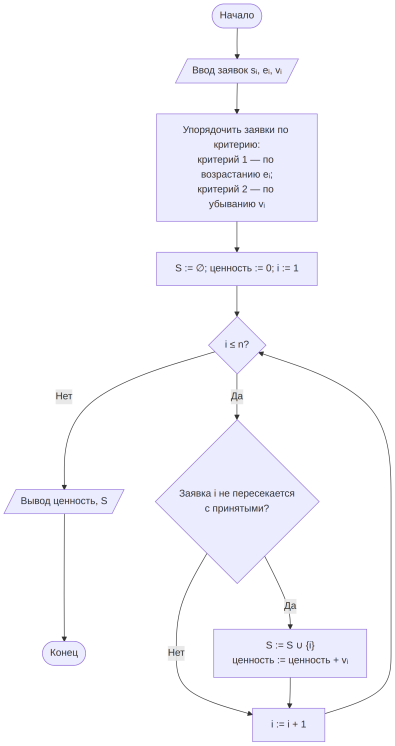
$$D[0] = 0, \qquad D[j] = \max\bigl(D[j-1],\ v_j + D[p(j)]\bigr).$$

Первая ветвь — заявка j не берётся; вторая — берётся, и тогда из остальных допустимы только заявки, закончившиеся не позже её начала. Ответ — $D[n]$; набор заявок восстанавливается обратным проходом. Трудоёмкость $\Theta(n \log n)$ (сортировка и n двоичных поисков). Для сверки реализован полный перебор — ветвление «взять / пропустить» по заявкам, упорядоченным по началу ($\Theta(2^n)$ в худшем случае).

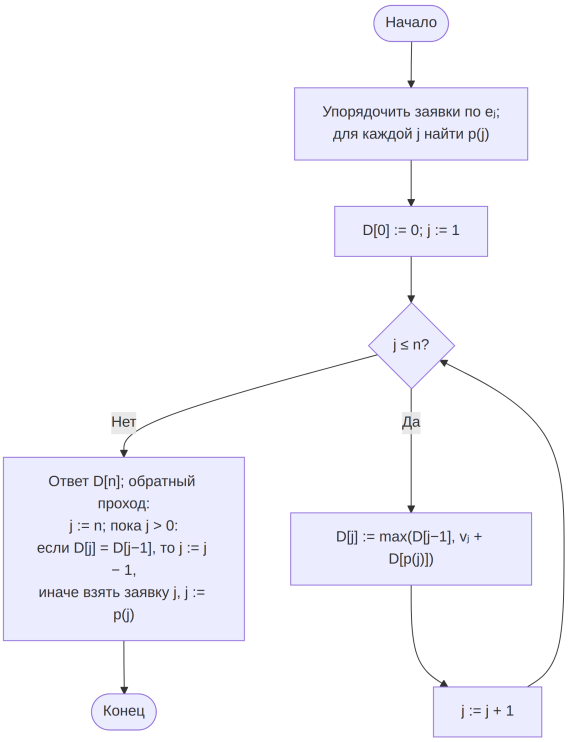
Качество жадного решения оценивается отношением $\rho = V_{\text{жадн}} / V_{\text{опт}} \le 1$ и отклонением $\delta = (1 - \rho) \cdot 100\,\%$.

3. Блок-схемы

3.1. Жадный алгоритм выбора заявок



3.2. Точный алгоритм (ДП)



4. Листинг программы

Программа содержит генерацию заявок, оба жадных критерия, точный алгоритм (ДП), полный перебор для проверки, серию случайных наборов, контрпримеры, замеры времени, прикладную задачу Ж7 и построение графиков.

```
"""
Лабораторная работа № 8. Вариант 14.
Выбор заявок: жадные критерии «раннее окончание» и «максимальная ценность заявки»
против точного решения (динамическое программирование). 35 заявок.
Дополнительное задание Ж7: распределение ресурса между инициативами, цена ошибки.

Заявка = (начало, окончание, ценность). Заявки совместимы, если не пересекаются:
начало следующей >= окончания предыдущей. Цель: максимизировать суммарную ценность.
"""
import bisect
import json
import random
import statistics
import time

N = 35                  # число заявок по варианту
HORIZON = 100           # горизонт планирования, ч
TRIALS = 500            # число случайных наборов
SEED = 2024


# ---------------------------------------------------------------- данные
def make_requests(n, rng, horizon=HORIZON):
    """n заявок: начало — случайный момент, длительность 3…20 ч, ценность 10…100."""
    reqs = []
    for _ in range(n):
        start = rng.randint(0, horizon - 3)
        end = min(horizon + 20, start + rng.randint(3, 20))
```

```
        reqs.append((start, end, rng.randint(10, 100)))
    return reqs


# ---------------------------------------------------- жадные алгоритмы
def greedy_select(reqs, key):
    """Жадный алгоритм: заявки упорядочиваются по критерию key (по возрастанию),
    заявка принимается, если не пересекается с уже принятыми.
    Принятые заявки хранятся упорядоченно по началу, проверяются только соседи."""
    order = sorted(range(len(reqs)), key=key)
    starts, chosen = [], []             # упорядоченные начала принятых заявок
    by_start = {}
    for i in order:
        s, e, v = reqs[i]
        pos = bisect.bisect_left(starts, s)
        ok_left = pos == 0 or reqs[by_start[starts[pos - 1]]][1] <= s
        ok_right = pos == len(starts) or reqs[by_start[starts[pos]]][0] >= e
        if ok_left and ok_right:
            starts.insert(pos, s)
            by_start[s] = i
            chosen.append(i)
    return sum(reqs[i][2] for i in chosen), sorted(chosen)


def greedy_earliest_end(reqs):
    """Критерий 1: раннее окончание (при равенстве — раннее начало)."""
    return greedy_select(reqs, key=lambda i: (reqs[i][1], reqs[i][0]))


def greedy_max_value(reqs):
    """Критерий 2: максимальная ценность заявки (при равенстве — раннее окончание)."""
    return greedy_select(reqs, key=lambda i: (-reqs[i][2], reqs[i][1]))


# ---------------------------------------------------- точные алгоритмы
def exact_dp(reqs, weighted=True):
    """Точное решение: взвешенный выбор заявок. Заявки упорядочены по окончанию,
    p[j] — число заявок, закончившихся не позже начала j-й.
    D[j] = max(D[j-1], v_j + D[p[j]]). Трудоёмкость θ(n log n)."""
    order = sorted(range(len(reqs)), key=lambda i: (reqs[i][1], reqs[i][0]))
    ends = [reqs[i][1] for i in order]
    w = [reqs[i][2] if weighted else 1 for i in order]
    p = [bisect.bisect_right(ends, reqs[i][0], 0, j) for j, i in enumerate(order)]
    D = [0] * (len(order) + 1)
    for j in range(1, len(order) + 1):
        D[j] = max(D[j - 1], w[j - 1] + D[p[j - 1]])
    chosen, j = [], len(order)
    while j > 0:                        # восстановление набора
        if D[j] == D[j - 1]:
            j -= 1
        else:
            chosen.append(order[j - 1])
            j = p[j - 1]
    return D[-1], sorted(chosen)


def exact_brute(reqs, weighted=True):
    """Полный перебор совместимых наборов (ветвление «взять / пропустить»).
    Возвращает (оптимум, число рассмотренных вершин перебора)."""
    order = sorted(range(len(reqs)), key=lambda i: reqs[i][0])
```

```
    best, nodes = 0, 0

    def rec(k, last_end, total):
        nonlocal best, nodes
        nodes += 1
        if k == len(order):
            best = max(best, total)
            return
        s, e, v = reqs[order[k]]
        rec(k + 1, last_end, total)                          # пропустить
        if s >= last_end:                                    # взять
            rec(k + 1, e, total + (v if weighted else 1))

    rec(0, -1, 0)
    return best, nodes


# ---------------------------------------------------------------- серии
def experiment(trials=TRIALS, n=N, seed=SEED):
    """Отклонение жадных решений от оптимума на серии случайных наборов."""
    rng = random.Random(seed)
    names = ["Раннее окончание", "Максимальная ценность"]
    dev = {k: [] for k in names}
    dev_count = {k: [] for k in names}
    hits = {k: 0 for k in names}
    hits_count = {k: 0 for k in names}
    losses = []
    for _ in range(trials):
        reqs = make_requests(n, rng)
        opt, _ = exact_dp(reqs)
        opt_cnt, _ = exact_dp(reqs, weighted=False)
        for name, func in zip(names, (greedy_earliest_end, greedy_max_value)):
            got, chosen = func(reqs)
            dev[name].append((opt - got) / opt * 100)
            hits[name] += got == opt
            cnt = len(chosen)
            dev_count[name].append((opt_cnt - cnt) / opt_cnt * 100)
            hits_count[name] += cnt == opt_cnt
        losses.append(opt - greedy_earliest_end(reqs)[0])
    return dev, hits, dev_count, hits_count


def verify_dp_against_brute(sets=200, n=15, seed=5):
    """Сверка ДП с полным перебором на малых наборах (обе постановки)."""
    rng = random.Random(seed)
    for _ in range(sets):
        reqs = make_requests(n, rng)
        assert exact_dp(reqs)[0] == exact_brute(reqs)[0]
        assert exact_dp(reqs, weighted=False)[0] == exact_brute(reqs, weighted=False)[0]
        assert greedy_earliest_end(reqs)[0] <= exact_dp(reqs)[0]
    return sets


def counterexamples():
    """Контрпримеры, построенные вручную."""
    ex1 = [(0, 5, 10), (1, 6, 100), (6, 9, 20)]              # ломает «раннее окончание»
    ex2 = [(0, 10, 50), (0, 5, 30), (5, 10, 30)]             # ломает «максимальную ценность»
    out = []
    for reqs in (ex1, ex2):
        g1, c1 = greedy_earliest_end(reqs)
```

```
        g2, c2 = greedy_max_value(reqs)
        opt, co = exact_dp(reqs)
        out.append({"reqs": reqs, "end": (g1, c1), "value": (g2, c2), "opt": (opt, co),
                    "count_opt": exact_dp(reqs, weighted=False)[0],
                    "count_value": len(c2), "count_end": len(c1)})
    return out


# ---------------------------------------------------------------- время
def timing():
    rng = random.Random(1)
    rows = []
    for n in (35, 100, 1000, 10_000, 100_000):
        reqs = make_requests(n, rng, horizon=HORIZON * n // N)     # плотность постоянна
        row = {"n": n}
       for name, func in (("greedy_end", greedy_earliest_end), ("greedy_value", greedy_max_value),
                          ("dp", exact_dp)):
            reps = 200 if n <= 1000 else 3
            t0 = time.perf_counter()
            for _ in range(reps):
                func(reqs)
            row[name] = (time.perf_counter() - t0) / reps
        rows.append(row)
    brute = []
    rng = random.Random(2)
    for n in (10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 70, 80):
        reqs = make_requests(n, rng)
        t0 = time.perf_counter()
        _, nodes = exact_brute(reqs)
        t = time.perf_counter() - t0
        brute.append({"n": n, "t": t, "nodes": nodes})
        if t > 10:
            break
    return rows, brute


# ---------------------------------------------------------------- Ж7: бюджет
def application_case(seed=14):
    """Одна производственная линия (единственный ресурс), 35 инициатив запрашивают
    интервалы её использования; ценность заявки — прибыль, тыс. руб."""
    rng = random.Random(seed)
    reqs = make_requests(N, rng)
    opt, c_opt = exact_dp(reqs)
    g1, c1 = greedy_earliest_end(reqs)
    g2, c2 = greedy_max_value(reqs)
    return {"reqs": reqs, "opt": (opt, c_opt), "end": (g1, c1), "value": (g2, c2)}


# ---------------------------------------------------------------- графики
def make_plots(res):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    fig, axes = plt.subplots(1, 2, figsize=(11, 4))
    for ax, (name, d) in zip(axes, res["dev"].items()):
        ax.hist(d, bins=30, color="#4c72b0" if name.startswith("Ран") else "#c44e52")
        ax.axvline(statistics.mean(d), color="k", ls="--", label=f"среднее
 ↪  {statistics.mean(d):.1f} %")
        ax.axvline(statistics.median(d), color="g", ls=":", label=f"медиана
 ↪  {statistics.median(d):.1f} %")
```

```
        ax.set_title(name); ax.set_xlabel("отклонение от оптимума, %"); ax.set_ylabel("число
↪  наборов")
        ax.legend(); ax.grid(True, alpha=.3)
    plt.tight_layout(); plt.savefig("img/lr8_hist.png", dpi=170); plt.close()

    t = res["timing"]
    plt.figure(figsize=(6.6, 4.2))
    ns = [r["n"] for r in t]
    for key, label, mk in (("greedy_end", "жадный (раннее окончание)", "o-"),
                           ("greedy_value", "жадный (макс. ценность)", "s-"),
                           ("dp", "точный (ДП)", "^-")):
        plt.loglog(ns, [r[key] for r in t], mk, label=label)
    plt.xlabel("число заявок n"); plt.ylabel("время, с"); plt.grid(True, which="both", alpha=.3)
    plt.title("Время работы алгоритмов"); plt.legend()
    plt.tight_layout(); plt.savefig("img/lr8_time.png", dpi=170); plt.close()

    fig, axes = plt.subplots(1, 2, figsize=(11, 3.8))
    for ax, ex, title in zip(axes, res["counter"], ("Контрпример 1 (критерий «раннее окончание»)",
                                                    "Контрпример 2 (критерий «максимальная
                                                     ↪  ценность»)")):
        for k, (s, e, v) in enumerate(ex["reqs"]):
            col = "#2ca02c" if k in ex["opt"][1] else "#bbbbbb"
            ax.barh(k, e - s, left=s, color=col, edgecolor="k")
            ax.text(s + (e - s) / 2, k, f"v={v}", ha="center", va="center", fontsize=9)
        ax.set_yticks(range(len(ex["reqs"])))
        ax.set_yticklabels([f"заявка {k + 1}" for k in range(len(ex["reqs"]))])
        ax.invert_yaxis(); ax.set_xlabel("время, ч"); ax.set_title(title, fontsize=10)
        ax.grid(True, axis="x", alpha=.3)
    plt.tight_layout(); plt.savefig("img/lr8_counter.png", dpi=170); plt.close()


# ---------------------------------------------------------------------- main
if __name__ == "__main__":
    res = {}
    print("Сверка ДП с перебором:", verify_dp_against_brute(), "наборов — совпало")

    res["counter"] = counterexamples()
    for ex in res["counter"]:
        print("Контрпример", ex["reqs"], "жадно(окончание)", ex["end"], "жадно(ценность)",
              ex["value"], "оптимум", ex["opt"])

    dev, hits, dev_c, hits_c = experiment()
    res["dev"], res["hits"], res["dev_count"], res["hits_count"] = dev, hits, dev_c, hits_c
    for name in dev:
        d = dev[name]
        print(f"{name:<24} среднее {statistics.mean(d):6.2f} медиана {statistics.median(d):6.2f} "
              f"макс {max(d):6.2f} точных {hits[name] / TRIALS * 100:5.1f} %  "
              f"| по числу заявок: среднее {statistics.mean(dev_c[name]):.2f} макс
               ↪  {max(dev_c[name]):.2f} "
              f"точных {hits_c[name] / TRIALS * 100:.1f} %")

    res["timing"], res["brute"] = timing()
    for r in res["timing"]:
        print(r)
    print(res["brute"])

    res["app"] = application_case()
    # цена ошибки по серии сценариев (ценность в тыс. руб.)
    rng = random.Random(SEED)
    loss_end, loss_val = [], []
```

```
    for _ in range(TRIALS):
        reqs = make_requests(N, rng)
        opt = exact_dp(reqs)[0]
        loss_end.append(opt - greedy_earliest_end(reqs)[0])
        loss_val.append(opt - greedy_max_value(reqs)[0])
    res["loss"] = {"end": loss_end, "value": loss_val}
    print("потери, тыс. руб.: окончание", statistics.mean(loss_end), max(loss_end),
          "ценность", statistics.mean(loss_val), max(loss_val))

    with open("res/lr8.json", "w", encoding="utf-8") as f:
        json.dump(res, f, ensure_ascii=False)
    make_plots(res)
```

5. Таблица результатов по серии наборов данных

Для каждого из 500 наборов вычислены оптимум (ДП) и оба жадных решения. Предварительно ДП сверен с полным перебором на 200 наборах по 15 заявок (обе постановки, совпадение на всех наборах).

Таблица 2. Отклонение жадных решений от оптимума по суммарной ценности (500 наборов, 35 заявок)

Критерий	Среднее отклонение, %	Медиана, %	Максимум, %	Доля точных совпадений
Раннее окончание	18,98	17,92	54,89	1,6 %
Максимальная ценность заявки	9,42	8,15	35,10	15,4 %

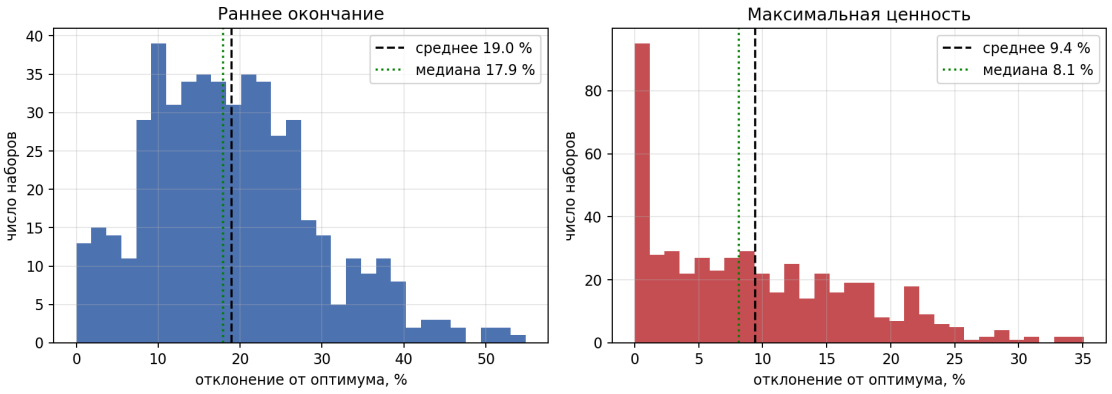


Рис. 1. Распределение отклонений жадных решений от оптимума

Критерий «раннее окончание» совпал с оптимумом лишь в 8 наборах из 500 (1,6 %), среднее отклонение 19,0 %, максимум 54,9 %: он не учитывает ценность заявок. Критерий «максимальная ценность» точнее (среднее 9,4 %, медиана 8,1 %, максимум 35,1 %, точных решений 15,4 %), но тоже далёк от гарантированного оптимума.

Проверка теории: постановка «максимальное число заявок».

Таблица 3. Отклонение по числу принятых заявок (500 наборов)

Критерий	Среднее отклонение числа заявок, %	Максимум, %	Доля точных совпадений
Раннее окончание	0,00	0,00	100,0 %
Максимальная ценность заявки	18,84	50,00	11,6 %

Критерий «раннее окончание» дал оптимальное число заявок на **всех** 500 наборах, что подтверждает теорему о его оптимальности (раздел 2.2). Критерий «максимальная

ценность» для этой цели непригоден: среднее отклонение 18,8 %, максимум 50 %, точных решений лишь 11,6 %. Значит, оптимальность жадности определяется не самой стратегией, а соответствием критерия целевой функции.

6. Контрпримеры, разобранные вручную

6.1. Контрпример для критерия «раннее окончание»

Заявка	Начало	Окончание	Ценность
1	0	5	10
2	1	6	100
3	6	9	20

Жадный алгоритм упорядочивает заявки по окончанию: 1 ($e = 5$), 2 ($e = 6$), 3 ($e = 9$). Принимается заявка 1 (ценность 10); заявка 2 начинается в момент $1 < 5$ и пересекается с ней — отвергается; заявка 3 начинается в момент $6 \geq 5$ — принимается. Итог: заявки 1, 3, ценность $10 + 20 = 30$.

Оптимум: заявки 2 и 3 (заявка 2 заканчивается в момент 6, заявка 3 начинается в момент 6 — совместимы), ценность $100 + 20 = 120$. Единственные совместимые пары: $\{1, 3\}$ (30) и $\{2, 3\}$ (120).

$$\rho = \frac{30}{120} = 0{,}25, \qquad \delta = 75\,\%.$$

Причина: критерий выбирает заявку, которая раньше заканчивается, и «не замечает» заявку 2, которая ценнее в десять раз.

6.2. Контрпример для критерия «максимальная ценность»

Заявка	Начало	Окончание	Ценность
1	0	10	50
2	0	5	30
3	5	10	30

Жадный алгоритм по ценности берёт заявку 1 (ценность 50); заявки 2 и 3 пересекаются с ней — отвергаются. Итог: 50. Оптимум: заявки 2 и 3 (не пересекаются: первая заканчивается в момент 5, вторая начинается в момент 5), ценность $30 + 30 = 60$.

$$\rho = \frac{50}{60} \approx 0{,}833, \qquad \delta \approx 16{,}7\,\%.$$

По числу заявок ошибка ещё заметнее: 1 заявка против 2 (отклонение 50 %). Причина: заявка 1 «дорогая», но занимает весь горизонт, и две более дешёвые заявки в сумме оказываются выгоднее. В этом примере критерий «раннее окончание» даёт оптимум (60).

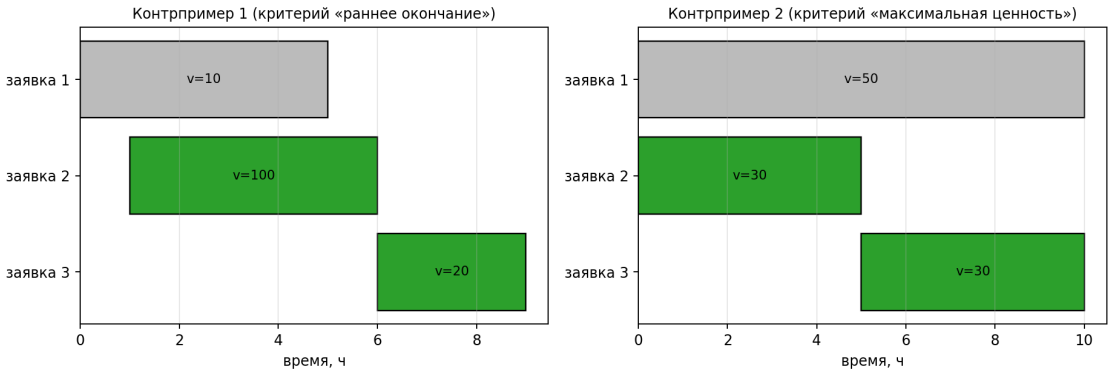


Рис. 2. Контрпримеры (зелёным отмечены заявки оптимального решения)

7. Сравнение времени работы алгоритмов

Размер задачи растёт при постоянной плотности заявок (горизонт увеличивается пропорционально n); время — среднее по повторам.

Таблица 4. Время работы жадных алгоритмов и точного ДП

n	Жадный (раннее окончание), с	Жадный (макс. ценность), с	Точный (ДП), с	ДП / жадный (окончание)
35	0,000015	0,000015	0,000021	1,41
100	0,000040	0,000039	0,000056	1,39
1 000	0,000515	0,000460	0,000505	0,98
10 000	0,005883	0,007648	0,007458	1,27
100 000	0,107014	0,193788	0,141943	1,33

Все три алгоритма имеют одинаковую асимптотику $\Theta(n \log n)$, поэтому точный алгоритм ДП медленнее жадного лишь на константный множитель (при $n = 100\,000$: 0,107 с — жадный по окончанию, 0,194 с — жадный по ценности, 0,142 с — ДП). Время перебора растёт экспоненциально:

Таблица 5. Время полного перебора (ветвление «взять / пропустить»)

n	Время полного перебора, с	Вершин перебора
10	0,0001	420
15	0,0004	2 109
20	0,0029	16 196
25	0,0113	66 733
30	0,0389	226 017
35	0,0370	329 545
40	0,0423	476 276
45	0,0798	806 324
50	0,3991	4 510 988
55	0,2918	2 986 188
60	0,5318	5 768 172
70	5,2231	60 432 944
80	43,7610	254 666 105

Для каждого n использован свой случайный набор, поэтому рост неравномерен (число вершин перебора зависит от числа пересечений). Перебор на 35 заявках занимает 0,037 с (329 545 вершин), на 70 — 5,2 с, на 80 — 43,8 с (254 666 105 вершин): **порог 10 с пройден между** $n = 70$ **и** $n = 80$. ДП уже при $n = 100$ работает около 0,03 мс (табл. 4).

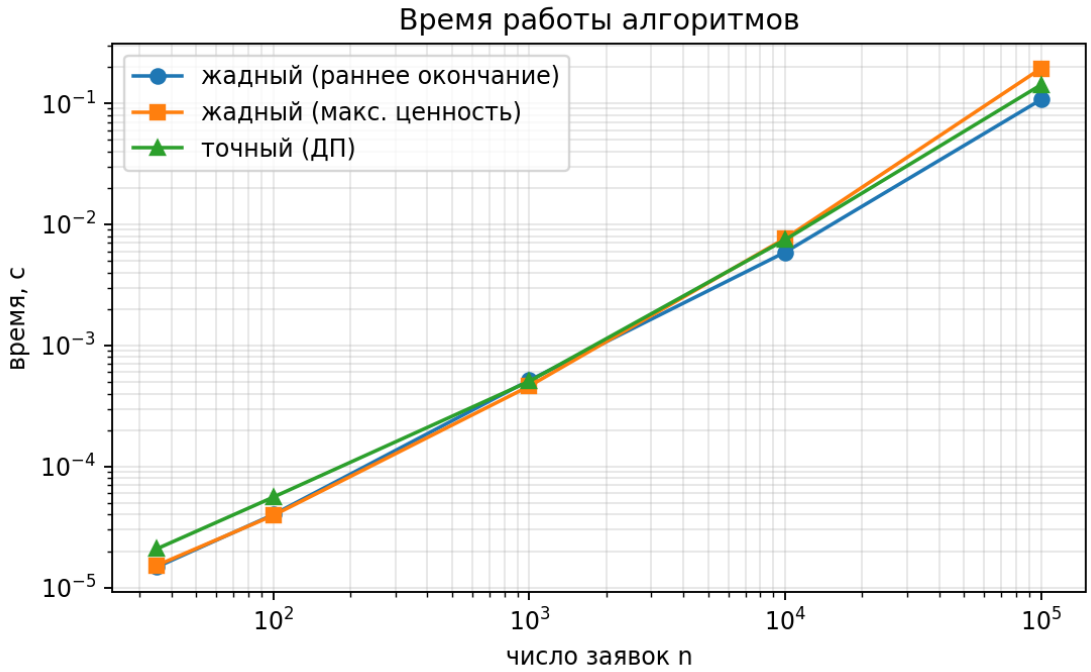


Рис. 3. Время работы жадных алгоритмов и ДП (логарифмические шкалы)

8. Дополнительное задание Ж7: распределение ресурса между инициативами

8.1. Постановка

Единственная производственная линия доступна 100 ч на цикл планирования. 35 инициатив (проектов) подали заявки на использование линии: каждая задаётся окном (начало, окончание, ч) и ожидаемой прибылью (тыс. руб.). Линия не может обслуживать две инициативы одновременно, поэтому нужно распределить ресурс (время линии) между инициативами так, чтобы суммарная прибыль была максимальной. Это в точности задача выбора заявок; ограниченным «бюджетом» здесь является время линии. Сопоставляются рекомендации трёх способов: точного (ДП) и двух жадных. Данные сгенерированы программой (фиксированное зерно).

8.2. Рекомендации алгоритмов на одном наборе заявок

Таблица 6. Итог для набора из 35 заявок

Способ	Принято заявок	Прибыль, тыс. руб.	Потеря к оптимуму, тыс. руб.
Точный (ДП)	8	571	0
Жадный: раннее окончание	8	451	120
Жадный: максимальная ценность	8	571	0

Точное решение принимает заявки из первой таблицы ниже, суммарная прибыль 571 тыс. руб. Жадный алгоритм по критерию «максимальная ценность» на этом наборе **совпал** с оптимумом, а критерий «раннее окончание» выбрал другой набор и потерял 120 тыс. руб. (21,0 % оптимума).

Оптимальный набор (ДП)

Заявка	Начало, ч	Окончание, ч	Прибыль, тыс. руб.
3	93	98	94
7	40	54	43

Заявка	Начало, ч	Окончание, ч	Прибыль, тыс. руб.
9	20	40	95
16	87	93	84
22	13	19	21
24	3	11	62
33	72	79	72
35	56	59	100

Набор жадного алгоритма «раннее окончание»

Заявка	Начало, ч	Окончание, ч	Прибыль, тыс. руб.
3	93	98	94
7	40	54	43
9	20	40	95
11	1	6	25
17	75	89	33
22	13	19	21
31	60	75	40
35	56	59	100

Цена ошибки на этом наборе: следование жадному правилу «раннее окончание» стоило бы **120 тыс. руб.** за цикл планирования.

8.3. Цена ошибки по серии сценариев

Для 500 сценариев (случайных наборов из 35 заявок) вычислена потеря прибыли относительно оптимума, тыс. руб. за цикл.

Таблица 7. Потеря прибыли при использовании жадного правила, тыс. руб. за цикл

Критерий	Средняя потеря	Медиана	90-й перцентиль	Максимум	Доля сценариев с потерей
Раннее окончание	118,3	113	205	304	98,4 %
Максимальная ценность	59,8	52	131	232	84,6 %

В среднем жадное правило по раннему окончанию теряет 118,3 тыс. руб. за цикл (90-й перцентиль — 205, максимум — 304), по максимальной ценности — 59,8 тыс. руб. (131 и 232). Если бы цикл планирования повторялся 12 раз в год (условное допущение), годовая потеря при правиле «раннее окончание» составила бы около 1,42 млн руб.

Рекомендация. Так как точное решение имеет ту же трудоёмкость $\Theta(n \log n)$, что и жадное (раздел 7), для распределения ресурса по прибыли следует применять ДП: приближённость жадного правила не окупается ни временем, ни простотой реализации. Критерий «раннее окончание» следует использовать только тогда, когда цель — максимальное число заявок (или все заявки равноценны), и тогда он оптимален.

9. Выводы

1. Жадная стратегия не гарантирует оптимума, если критерий отбора не соответствует целевой функции. Для взвешенного выбора заявок критерий «раннее окончание» совпал с оптимумом лишь на 1,6 % наборов (среднее отклонение 19,0 %, максимум 54,9 %), критерий «максимальная ценность» — на 15,4 % (среднее 9,4 %, максимум 35,1 %).
2. Для постановки «максимальное число заявок» критерий «раннее окончание» оптимален: на всех 500 наборах отклонение равно нулю, что подтверждает теорию. Критерий «максимальная ценность» в этой постановке даёт отклонение до 50 %. Следовательно, оптимальность жадности — свойство пары «задача — критерий», которое требует доказательства.

3. Контрпримеры показывают механизмы ошибок: «раннее окончание» отвергает очень ценную заявку ради менее ценной, но более короткой (отклонение 75 %); «максимальная ценность» занимает весь горизонт одной дорогой заявкой, вытесняя две, суммарно более выгодные (отклонение 16,7 %).
4. Точный алгоритм (ДП с двоичным поиском) имеет ту же трудоёмкость $\Theta(n \log n)$, что и жадный, и работает практически так же быстро; полный перебор экспоненциален и неприменим уже при $n \approx 80$ (порог 10 с). Поэтому в данной задаче нет причин довольствоваться приближённым решением.
5. В прикладной задаче распределения ресурса цена ошибки измерима: в среднем 118,3 тыс. руб. за цикл для критерия «раннее окончание» и 59,8 тыс. руб. для критерия «максимальная ценность». Рекомендация: использовать точный алгоритм; жадное правило применять, когда оно доказано оптимальным (максимальное число заявок) либо когда решение должно приниматься в реальном времени без возможности вычисления ДП.

10. Ответы на контрольные вопросы

1. Опишите общую схему жадного алгоритма. Чем он отличается от динамического программирования? Элементы упорядочиваются по некоторому критерию, затем просматриваются по очереди и включаются в решение, если это допустимо; принятые решения не пересматриваются. Трудоёмкость определяется сортировкой, $\Theta(n \log n)$. ДП, напротив, рассматривает все подзадачи и выбирает лучший вариант из их решений (таблица), поэтому даёт точный оптимум, но обычно дороже. В работе: жадный алгоритм выбора заявок — один проход по отсортированным заявкам; ДП — рекуррентное соотношение $D[j] = \max(D[j-1], v_j + D[p(j)])$.

2. Сформулируйте свойство жадного выбора и свойство оптимальной подструктуры. Свойство жадного выбора: существует оптимальное решение, содержащее выбор, сделанный жадно на первом шаге. Оптимальная подструктура: после первого выбора остаётся подзадача того же вида, и оптимум складывается из выбора и оптимума подзадачи. Для максимального числа заявок: оптимальное решение содержит заявку с наименьшим окончанием (замена первой заявки), а остаток — та же задача на заявках, начинающихся не раньше e_1.

3. Почему в дробном рюкзаке жадность оптимальна, а в задаче 0/1 — нет? В дробном рюкзаке предмет делится, поэтому вместимость всегда заполняется полностью, и достаточно брать предметы по убыванию удельной ценности. В задаче 0/1 предмет неделим, жадный выбор может оставить неиспользуемый остаток: при $W = 10$ и предметах $(6; 12)$, $(5; 9)$, $(5; 9)$ жадный алгоритм берёт первый предмет (12), оптимум — два других (18), отклонение 33 %. Аналогичный механизм в нашей работе: дорогая заявка занимает ресурс, и две заявки меньшей ценности в сумме выгоднее (раздел 6.2).

4. Приведите контрпример к жадному размену для системы номиналов 1, 3, 4. Сумма $S = 6$: жадный алгоритм даёт $4 + 1 + 1$ (три монеты), оптимум — $3 + 3$ (две монеты). Аналогично, для системы 1, 5, 12 (ЛР № 7) жадный алгоритм ошибается на сумме 15: $12 + 1 + 1 + 1$ (4 монеты) против $5 + 5 + 5$ (3 монеты).

5. Для каких систем номиналов жадный размен оптимален? Как это проверить? Для «канонических» систем, например 1, 5, 10, 25 или 1, 2, 5, 10. Простого критерия нет: проверяют сравнением жадного результата с ДП. Известный результат (Козен и Закс) утверждает, что достаточно проверить суммы, не превосходящие $c_{k-1} + c_k$ (сумма двух старших номиналов): если на них жадный алгоритм не ошибается, то он оптимален для любой суммы. Для системы 1, 5, 12 проверка идёт до 17, и ошибка найдена уже на 15.

6. Что такое коэффициент качества приближённого решения ρ? Отношение $\rho = V_{\text{жадн}} / V_{\text{опт}} \le 1$ для задач максимизации; $\delta = (1 - \rho) \cdot 100\,\%$ — относительное отклонение. В работе: контрпример 1 — $\rho = 0{,}25$, контрпример 2 — $\rho \approx 0{,}83$; по серии из 500 наборов средние ρ равны примерно $1 - 0{,}190 = 0{,}81$ для критерия «раннее окончание» и $1 - 0{,}094 = 0{,}91$ для критерия «максимальная ценность».

7. Почему жадный алгоритм для задачи о выборе заявок оптимален? Какой критерий используется? Для задачи о максимальном числе непересекающихся заявок оптимален критерий «раннее окончание»: заявка, заканчивающаяся раньше всех, оставляет наибольшее время для остальных, и любое оптимальное решение можно

преобразовать так, чтобы оно содержало эту заявку (обмен). Экспериментально: 500 из 500 наборов совпали с оптимумом (таблица 3). Но для **взвешенной** задачи (разные ценности) это уже не так: отклонение до 54,9 %.

8. Как оценить, оправдано ли применение приближённого алгоритма в конкретной задаче? Сопоставить потери качества с выигрышем во времени и простоте: измерить на серии данных отклонение (среднее, максимум, доля точных решений), цену ошибки в денежном выражении и время точного алгоритма. В нашей задаче точный алгоритм так же быстр ($\Theta(n \log n)$), а средняя потеря жадного правила — 118,3 и 59,8 тыс. руб. за цикл, поэтому приближённое решение не оправдано.

9. Сравните трудоёмкость жадного алгоритма, ДП и полного перебора для рюкзака. Жадный — $\Theta(n \log n)$, память $O(n)$, результат приближённый; ДП — $\Theta(nW)$, память $\Theta(nW)$ или $\Theta(W)$, точный, но эффективен лишь при умеренных W; полный перебор — $\Theta(2^n)$, точный, применим лишь при малых n. В нашей задаче выбора заявок: жадный и ДП — $\Theta(n \log n)$ ($n = 10^5$: около 0,1–0,2 с), перебор — экспонента (при $n = 80$ — 43,8 с).

10. Приведите пример управленческого решения, которое принимается по жадному правилу. Каковы риски такого подхода? Например, «сначала финансируем самые рентабельные проекты» или «принимаем заказы с ближайшим сроком». Риски: неиспользованный остаток ресурса, игнорирование взаимовлияния решений, потери по сравнению с оптимумом (в нашем опыте в среднем 118,3 тыс. руб. за цикл при выборе по раннему окончанию), отсутствие гарантий качества. Поэтому величину потерь следует оценивать на данных, а не постулировать.